

Python Programming

Day 11: Comprehensive Practice

Basics, Functions, Data Structures, OOP, NumPy, Pandas, Matplotlib

Contents

1	학습 목표	2
2	Basic Python	2
2.1	최종 코드: 01_basic.py	2
3	Functions	3
3.1	최종 코드: 02_functions.py	3
4	Data Structures	5
4.1	최종 코드: 03_data_structures.py	5
5	Object-Oriented Programming	7
5.1	최종 코드: 04_oop.py	7
6	NumPy	8
6.1	최종 코드: 05_numpy.py	8
7	Pandas	9
7.1	최종 코드: 06_pandas.py	9
8	Matplotlib	10
8.1	최종 코드: 07_matplotlib.py	10
9	종합 실습 파일 목록	11
10	Day 11 핵심 요약	12

1. 학습 목표

학습 목표

Day 11은 Python Day 1-10에서 학습한 내용을 새로운 개념 없이 종합적으로 복습하는 과정이다. 이 장을 마치면 다음 내용을 직접 코드로 작성할 수 있다.

- 조건문, 반복문, 리스트, 딕셔너리를 이용해 데이터를 처리하기
- 기능을 함수로 분리하고 매개변수와 반환값을 활용하기
- 중첩된 리스트와 딕셔너리를 탐색하고 수정하기
- 클래스, 객체, 상속, 오버라이딩을 이용해 데이터를 구조화하기
- NumPy 배열의 인덱싱, 슬라이싱, 배열 연산을 수행하기
- Pandas DataFrame을 생성하고 데이터를 선택, 수정, 분석하기
- Matplotlib을 이용해 데이터를 여러 형태의 그래프로 시각화하기

핵심 포인트

Day 11의 목적은 새로운 문법을 추가하는 것이 아니라 이미 학습한 Python 기능을 직접 다시 구현하면서 기본기를 점검하는 것이다.

2. Basic Python

첫 번째 실습에서는 학생 점수 데이터를 이용해 Python의 기본 자료구조와 제어문을 복습한다. `sum()`, `max()`, `min()`, `sorted()`에 의존하지 않고 반복문과 조건문으로 직접 처리한다.

2.1. 최종 코드: 01_basic.py

```

1 students = [
2     {"name": "Alice", "score": 85},
3     {"name": "Bob", "score": 72},
4     {"name": "Charlie", "score": 93},
5     {"name": "David", "score": 68},
6     {"name": "Emma", "score": 88}
7 ]
8
9 total = 0
10
11 for i in range(0, len(students)):
12     print(f'{students[i]["name"]}: {students[i]["score"]}')
13     total += students[i]["score"]
14
15 print()
16
17 average = total / len(students)
18
19 print(f"Total: {total}")
20 print(f"Average: {average}")
21 print()
22
23 print("Students with score >= 80")
24

```

```

25 for i in range(0, len(students)):
26     if students[i]["score"] >= 80:
27         print(f'{students[i]["name"]}: {students[i]["score"]}')
28
29 print()
30
31 top_index = 0
32
33 for i in range(1, len(students)):
34     if students[i]["score"] > students[top_index]["score"]:
35         top_index = i
36
37 print(f'Top Student: {students[top_index]["name"]}')
38 print(f'Score: {students[top_index]["score"]}')
39 print()
40
41 for i in range(0, len(students)):
42     if students[i]["score"] >= 90:
43         grade = "A"
44     elif students[i]["score"] >= 80:
45         grade = "B"
46     elif students[i]["score"] >= 70:
47         grade = "C"
48     elif students[i]["score"] >= 60:
49         grade = "D"
50     else:
51         grade = "F"
52
53     print(f'{students[i]["name"]}: {grade}')

```

핵심 포인트

최댓값을 직접 찾을 때는 첫 번째 원소를 현재 최고값으로 설정하고 두 번째 원소부터 비교하며 더 큰 값이 나오면 갱신할 수 있다.

3. Functions

두 번째 실습에서는 기본 데이터 처리 로직을 함수로 분리한다. 함수 내부에서 모든 결과를 출력하기 보다 필요한 값을 return으로 반환하여 호출한 코드에서 재사용한다.

3.1. 최종 코드: 02_functions.py

```

1 students = [
2     {"name": "Alice", "score": 85},
3     {"name": "Bob", "score": 72},
4     {"name": "Charlie", "score": 93},
5     {"name": "David", "score": 68},
6     {"name": "Emma", "score": 88}
7 ]
8
9
10 def calculate_total(students):
11     total = 0
12
13     for student in students:
14         total += student["score"]
15

```

```
16     return total
17
18
19 def calculate_average(students):
20     return calculate_total(students) / len(students)
21
22
23 def get_high_scorers(students, cutoff):
24     result = []
25
26     for student in students:
27         if student["score"] >= cutoff:
28             result.append(student)
29
30     return result
31
32
33 def get_top_student(students):
34     top_index = 0
35
36     for i in range(1, len(students)):
37         if students[i]["score"] > students[top_index]["score"]:
38             top_index = i
39
40     return students[top_index]
41
42
43 def get_grade(score):
44     if score >= 90:
45         return "A"
46     elif score >= 80:
47         return "B"
48     elif score >= 70:
49         return "C"
50     elif score >= 60:
51         return "D"
52     else:
53         return "F"
54
55
56 print(f"Total: {calculate_total(students)}")
57 print(f"Average: {calculate_average(students)}")
58 print()
59
60 print("Students with score >= 80")
61
62 for student in get_high_scorers(students, 80):
63     print(f'{student["name"]}: {student["score"]}')
64
65 print()
66
67 top_student = get_top_student(students)
68
69 print(f'Top Student: {top_student["name"]}')
70 print(f'Score: {top_student["score"]}')
71 print()
72
73 print("Grades")
74
75 for student in students:
```

```
76 print(f'{student["name"]}: {get_grade(student["score"])}')
```

핵심 포인트

함수의 결과를 여러 번 사용할 경우 같은 함수를 반복 호출하기보다 반환값을 변수에 저장하여 재사용할 수 있다.

4. Data Structures

세 번째 실습에서는 리스트 안에 딕셔너리가 있고, 그 딕셔너리 안에 다시 리스트와 딕셔너리가 들어 있는 중첩 자료구조를 다룬다. 학생 중심 데이터를 탐색하고 수정한 뒤 과목 중심 데이터로 다시 구성한다.

4.1. 최종 코드: 03_data_structures.py

```
1 students = [
2     {
3         "name": "Alice",
4         "major": "Physics",
5         "courses": ["Python", "Math"],
6         "scores": {"Python": 88, "Math": 92}
7     },
8     {
9         "name": "Bob",
10        "major": "Engineering",
11        "courses": ["Python", "C++"],
12        "scores": {"Python": 76, "C++": 85}
13    },
14    {
15        "name": "Charlie",
16        "major": "Physics",
17        "courses": ["Math", "C++"],
18        "scores": {"Math": 95, "C++": 91}
19    },
20    {
21        "name": "David",
22        "major": "Engineering",
23        "courses": ["Python", "Math", "C++"],
24        "scores": {"Python": 82, "Math": 79, "C++": 88}
25    }
26 ]
27
28 print("Physics Students")
29
30 for student in students:
31     if student["major"] == "Physics":
32         print(f'{student["name"]}: {student["courses"]}')
33
34 print()
35
36 print("Python Students")
37
38 for student in students:
39     for course in student["courses"]:
40         if course == "Python":
41             print(f'{student["name"]}: {student["scores"]["Python"]}')
```

```
42
43 print()
44
45 for student in students:
46     if student["name"] == "Bob":
47         student["courses"].append("Math")
48         student["scores"]["Math"] = 84
49         print(student)
50
51 print()
52
53 new_student = {
54     "name": "Emma",
55     "major": "Physics",
56     "courses": ["Python", "C++", "Math"],
57     "scores": {"Python": 94, "C++": 90, "Math": 89}
58 }
59
60 students.append(new_student)
61
62 for student in students:
63     print(student["name"])
64
65 print()
66
67 cpp_list = []
68
69 for student in students:
70     for course in student["courses"]:
71         if course == "C++":
72             cpp_list.append(
73                 {
74                     "name": student["name"],
75                     "score": student["scores"]["C++"]
76                 }
77             )
78
79 top_index = 0
80
81 for i in range(1, len(cpp_list)):
82     if cpp_list[i]["score"] > cpp_list[top_index]["score"]:
83         top_index = i
84
85 print(f'Top C++ Student: {cpp_list[top_index]["name"]}')
86 print(f'Score: {cpp_list[top_index]["score"]}')
87 print()
88
89 course_students = {}
90 py_group = []
91 cpp_group = []
92 math_group = []
93
94 for student in students:
95     for course in student["courses"]:
96         if course == "Python":
97             py_group.append(student["name"])
98         elif course == "C++":
99             cpp_group.append(student["name"])
100         elif course == "Math":
101             math_group.append(student["name"])
```

```

102 course_students["Python"] = py_group
103 course_students["Math"] = math_group
104 course_students["C++"] = cpp_group
105
106
107 print(course_students)

```

핵심 포인트

중첩 자료구조에서는 현재 접근하고 있는 값이 리스트인지 딕셔너리인지 구분하는 것이 중요하다. 리스트에는 인덱스와 `append()`를 사용하고, 딕셔너리에는 `key`를 이용해 값을 읽거나 추가할 수 있다.

5. Object-Oriented Programming

네 번째 실습에서는 영화 데이터를 클래스로 표현한다. 기본 클래스와 자식 클래스를 만들고, 객체들을 하나의 리스트에 저장하여 상속과 오버라이딩을 복습한다.

5.1. 최종 코드: 04_oop.py

```

1 class Movie:
2     def __init__(self, title, year, rating):
3         self.title = title
4         self.year = year
5         self.rating = rating
6
7     def show_info(self):
8         print(f"Title: {self.title}")
9         print(f"Year: {self.year}")
10        print(f"Rating: {self.rating}")
11
12    def is_high_rating(self):
13        return self.rating >= 9.0
14
15
16 movies = []
17
18 movies.append(Movie("Interstellar", 2014, 9.1))
19 movies.append(Movie("Inception", 2010, 8.8))
20 movies.append(Movie("Parasite", 2019, 8.6))
21 movies.append(Movie("The Dark Knight", 2008, 9.0))
22
23 for movie in movies:
24     movie.show_info()
25     print()
26
27 print("High Rated Movies")
28
29 for movie in movies:
30     if movie.is_high_rating():
31         print(movie.title)
32
33 print()
34
35
36 class StreamingMovie(Movie):

```



```

37     def __init__(self, title, year, rating, platform):
38         super().__init__(title, year, rating)
39         self.platform = platform
40
41     def show_info(self):
42         super().show_info()
43         print(f"Platform: {self.platform}")
44
45 movies.append(StreamingMovie("Dune", 2021, 8.5, "Netflix"))
46 movies.append(StreamingMovie("Oppenheimer", 2023, 8.9, "Watcha"))
47
48 for movie in movies:
49     movie.show_info()
50     print()
51
52 top_rating_movie = movies[0]
53
54 for i in range(1, len(movies)):
55     if movies[i].rating > top_rating_movie.rating:
56         top_rating_movie = movies[i]
57
58 print("Top Movie")
59 print(f"Title: {top_rating_movie.title}")
60 print(f"Rating: {top_rating_movie.rating}")
61

```

핵심 포인트

부모 클래스의 생성자와 메서드는 `super()`를 이용해 재사용할 수 있다. 같은 `show_info()` 호출이라도 실제 객체가 자식 클래스의 객체라면 오버라이딩된 메서드가 실행된다.

6. NumPy

다섯 번째 실습에서는 NumPy 배열 생성, 인덱싱, 슬라이싱, 원소별 배열 연산과 2차원 배열 접근을 복습한다.

6.1. 최종 코드: 05_numpy.py

```

1 import numpy as np
2
3
4 a = np.array([10, 20, 30, 40, 50])
5 b = np.array([2, 4, 6, 8, 10])
6
7 matrix = np.array([
8     [1, 2, 3],
9     [4, 5, 6],
10    [7, 8, 9]
11 ])
12
13 print(a[0])
14 print(a[-1])
15 print(a[2])
16 print()
17
18 print(a[1:4])
19 print()

```

```

20
21 print(a + b)
22 print(a - b)
23 print(a * b)
24 print(a / b)
25 print()
26
27 print(a + 5)
28 print()
29
30 print(b * 3)
31 print()
32
33 print(matrix[1, 1])
34 print()
35
36 print(matrix[1, :])
37 print()
38
39 print(matrix * 2)
40 print(matrix + 10)
41 print()
42
43 print(np.zeros(5))
44 print(np.ones(5))
45 print(np.arange(0, 10, 2))
46 print(np.linspace(0, 1, 5))

```

핵심 포인트

NumPy 배열에 대한 +, -, *, / 연산은 같은 위치의 원소에 대해 수행된다. 스칼라와 배열을 연산하면 해당 연산이 배열의 모든 원소에 적용된다.

7. Pandas

여섯 번째 실습에서는 딕셔너리로부터 DataFrame을 생성하고 열과 행을 선택한 뒤 새로운 열을 추가하고 기본 통계를 확인한다.

7.1. 최종 코드: 06_pandas.py

```

1 import pandas as pd
2
3
4 data = {
5     "Name": ["Alice", "Bob", "Charlie", "David", "Emma"],
6     "Physics": [88, 76, 95, 82, 91],
7     "Math": [92, 85, 90, 78, 94]
8 }
9
10 df = pd.DataFrame(data)
11
12 print(df)
13 print()
14
15 print(df["Physics"])
16 print(df["Name"])
17 print()

```

```

18
19 print(df.loc[2])
20 print(df.loc[1:3, ["Name", "Physics"]])
21 print()
22
23 print(df.iloc[0])
24 print(df.iloc[-1])
25 print()
26
27 df["Average"] = (df["Physics"] + df["Math"]) / 2
28
29 print(df)
30 print()
31
32 print(f"Average of Physics: {df['Physics'].mean()}")
33 print(f"Average of Math: {df['Math'].mean()}")
34 print(f"Maximum of Physics: {df['Physics'].max()}")
35 print(f"Minimum of Math: {df['Math'].min()}")
36 print()
37
38 print(df.describe())

```

핵심 포인트

loc는 label을 기준으로 데이터를 선택하고 iloc는 정수 위치를 기준으로 선택한다. DataFrame의 열끼리 직접 연산하여 새로운 열을 만들 수도 있다.

8. Matplotlib

일곱 번째 실습에서는 NumPy 배열을 Matplotlib에 전달하여 선 그래프, 산점도, 막대그래프를 그린다. 제목, 축 이름, 범례, 격자를 추가하여 기본적인 데이터 시각화 흐름을 복습한다.

8.1. 최종 코드: 07_matplotlib.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 x = np.array([1, 2, 3, 4, 5])
6
7 physics = np.array([70, 75, 83, 88, 95])
8 math = np.array([65, 72, 80, 85, 90])
9
10 plt.plot(x, physics)
11 plt.title("Physics Scores")
12 plt.xlabel("Test")
13 plt.ylabel("Score")
14 plt.show()
15
16 plt.plot(x, physics, label="Physics")
17 plt.plot(x, math, label="Math")
18 plt.title("Physics vs Math")
19 plt.xlabel("Test")
20 plt.ylabel("Score")
21 plt.grid(True)
22 plt.legend()
23 plt.show()

```

```

24 plt.scatter(physics, math)
25 plt.title("Physics vs Math Scores")
26 plt.xlabel("Physics Scores")
27 plt.ylabel("Math Scores")
28 plt.show()
29
30
31 students = ["Alice", "Bob", "Charlie", "David", "Emma"]
32
33 plt.bar(students, physics)
34 plt.title("Physics Scores by Student")
35 plt.xlabel("Student")
36 plt.ylabel("Physics Score")
37 plt.show()
38
39 x = np.linspace(0, 10, 100)
40 y = x ** 2
41
42 plt.plot(x, y)
43 plt.title("y = x^2")
44 plt.xlabel("x")
45 plt.ylabel("y")
46 plt.grid(True)
47 plt.show()

```

핵심 포인트

NumPy는 수치 데이터를 생성하고 계산하는 역할을 담당하며 Matplotlib은 계산된 데이터를 그래프로 표현할 수 있다. `plt.show()`를 각 그래프 뒤에 호출하면 그래프를 하나씩 분리하여 확인할 수 있다.

9. 종합 실습 파일 목록

종합 실습

Day 11의 실습 파일은 다음과 같이 관리한다.

- 01_basic.py
- 02_functions.py
- 03_data_structures.py
- 04_oop.py
- 05_numpy.py
- 06_pandas.py
- 07_matplotlib.py
- python_day11.tex
- python_day11.pdf

10. Day 11 핵심 요약

- 리스트와 딕셔너리는 여러 데이터를 구조적으로 저장하고 반복문으로 처리할 수 있다.
- 조건문을 이용하면 데이터의 값에 따라 서로 다른 처리를 수행할 수 있다.
- 최댓값을 직접 찾을 때는 하나의 값을 현재 최고값으로 두고 순회하면서 갱신할 수 있다.
- 함수는 반복되는 기능을 분리하고 매개변수를 통해 데이터를 전달받으며 `return`으로 결과를 반환한다.
- 함수의 반환값은 변수에 저장하여 여러 번 재사용할 수 있다.
- 중첩 자료구조에서는 각 단계의 자료형을 구분하며 접근해야 한다.
- 클래스는 데이터와 관련 기능을 하나의 객체로 묶을 수 있다.
- `__init__()`은 객체가 생성될 때 인스턴스 변수를 초기화한다.
- 상속을 이용하면 기존 클래스의 기능을 재사용하고 새로운 기능을 추가할 수 있다.
- 메서드 오버라이딩을 이용하면 자식 클래스에서 부모 클래스의 동작을 변경하거나 확장할 수 있다.
- NumPy 배열은 인덱싱과 슬라이싱을 지원하며 배열 전체에 효율적으로 수치 연산을 적용할 수 있다.
- 2차원 NumPy 배열은 행과 열의 인덱스를 이용해 원하는 원소와 부분 배열에 접근할 수 있다.
- Pandas DataFrame은 행과 열 형태의 데이터를 다루기 위한 자료구조이다.
- `loc`와 `iloc`를 이용하면 원하는 행과 열을 선택할 수 있다.
- DataFrame의 열끼리 연산하여 새로운 열을 만들고 `mean()`, `max()`, `min()`, `describe()`로 데이터를 분석할 수 있다.
- Matplotlib의 `plot()`, `scatter()`, `bar()`를 이용해 여러 형태의 그래프를 만들 수 있다.
- NumPy로 생성하거나 계산한 데이터를 Matplotlib으로 시각화할 수 있다.
- Day 11은 Python Day 1-10에서 학습한 내용을 영역별 종합 실습으로 다시 확인하는 과정이다.